Context engineering in practice

Make the team knowledge reusable, relevant and verifiable.

THE MAIN QUESTION

How do I make the right team knowledge available without repeating it in every prompt?

Standing team rules

Teach Copilot the fictional TAO standard.

A reusable workflow

Repair a skill, then build DeliveryVehicle CRUD.

F1: ask clearly. F2: define the role. F3: supply the right knowledge and procedure.

A clear request still needs the right evidence

Prompt engineering shapes the request. Context engineering shapes what is available to answer it.

Prompt engineering

Goal, wording, constraints and the output you want.

Context engineering

The evidence, rules, tools and resources the model can use.

EXPLICIT

Files, selections, screenshots and attachments you supply.

PERSISTENT

Repository docs, instructions and reusable skills.

DISCOVERED

Workspace search, retrieved docs
and tool results.

Do not assume it read the whole repo. Point to key files and inspect what was actually used.

Relevant context beats a bigger chat

Instructions, history, files and tool results share a finite context window.

Finite budget

Include what the task needs.

Long history can be summarized; important details can be lost.

Noise compounds

Stale docs and contradictory rules can repeatedly mislead the agent.

More text is not more certainty.

Fresh comparisons

Start a new chat for before/after tests.

Carry key decisions explicitly; chat history is not permanent team policy.

Use approved data. Review third-party instructions and scripts; retrieved content is not automatically authoritative.

Put standing project knowledge in one place

Keep shared guidance factual, concise and reviewable through Git.

`.github/copilot-instructions.md`

```
# OctoCAT project guidance
- React + Express + SQLite; TypeScript.
- Install: make install.
- Run API: make dev-api.
- API tests: make test-api (watch; q exits).
- Keep routes thin; use repositories.
- Use shared errors and parameterized SQL.
- Keep Swagger aligned with behavior.
- Architecture: docs/architecture.md
```

Include

Purpose, commands and conventions.

Link

Architecture and deeper documentation.

Keep out

Secrets and temporary ticket notes.

Maintain

Verify commands and file paths.

No frontmatter needed. These rules guide chat and agents, not grey-text completions.

SCOPED INSTRUCTIONS

Give each part of the codebase its own guidance

Use file-pattern targeting instead of sending API rules with every UI task.

`.github/instructions/api.instructions.md`

```
---
applyTo: "api/src/**/*.ts"
---
- Use parameterized SQL.
- Use shared error classes.
- Keep Swagger in route comments.
```

EXAMPLE FILE PATTERNS

API

`api/src/**/*.ts`

Frontend

`frontend/src/**`

Database

`api/database/**`

Project-wide and matching scoped guidance can both apply. Avoid conflicts and test globs against real paths.

Be concrete, sourced and explicit about limits

A model can know the framework without knowing your internal convention.

VAGUE

Use the best logging library.
Follow team standards.

Which library? Which standards? What is out of scope?

CONCRETE + SOURCED

Use TAO for logging and metrics.
Read docs/tao.md for conventions.
Never install it or add it to package.json.
Do not introduce another logging stack.

TAO IS FICTIONAL: Ask-only comparison. Never install @tao/core or write code that depends on it.

Four building blocks, four different jobs

Choose the smallest reusable unit that solves the actual repetition.

Instructions

Standing rules. Project-wide or scoped.

```
copilot-instructions.md
*.instructions.md
```

"Use TAO for logging."

Prompt files

Reusable requests you invoke.

```
*.prompt.md
```

"Refine this prompt."

Agents

Roles, workflows and available capabilities.

```
*.agent.md
```

"A reviewer without edit tools."

Skills

Task procedures with supporting resources.

```
SKILL.md + resources
```

"Add an API entity end to end."

They compose: your agent follows team rules and can load a skill while handling your request.

Package the procedure, not just a long prompt

A skill is a folder: metadata, a workflow and the resources that workflow needs.

```
.github/skills/api-endpoint/
|-- SKILL.md
|   |-- name + description
|   `-- workflow + checklist
`-- references/
    |-- database-conventions.md
    |-- error-handling.md
    |-- swagger-patterns.md
    `-- testing-patterns.md
```

Skills can also include scripts, templates and examples. Review them before reuse.

1 DISCOVER THE WHEN

Name and description identify relevant tasks.

2 LOAD THE HOW

Read the workflow when the skill is activated.

3 LOAD NEEDED DETAIL

Read linked resources or reviewed scripts as needed.

Skills do not themselves grant tool permissions.

A useful workflow needs a discoverable trigger

The body can be excellent and still be unavailable if its discovery metadata is missing.

```
---
name: api-endpoint
description: >-
  Create or extend OctoCAT API entities.
  Use for CRUD routes, repositories, migrations,
  Swagger documentation and tests.
---
```

WEAK DESCRIPTION

"API helper."

BETTER DESCRIPTION

Say the task and its triggers: new entities, CRUD, routes, repositories, migrations, Swagger and tests.

Match name to the folder; use SKILL.md.
Save, confirm discovery, then test selection.

A listed skill can also be invoked through / where supported. Test auto-selection separately.

Loaded is not followed. Followed is not verified.

Inspect the evidence at each stage, not just the presence of a reference.

Listed

Visible in the configuration UI.

Included

References and skill/tool activity.

Followed

Work matches the rules and workflow.

Verified

Tests and actual API behavior pass.

MAINTAIN THE SOURCE

Verify commands, paths and rules.
Resolve conflicts against actual code.
Review context changes like code.

CHECK THE WHOLE FEATURE

Model + repository; route + registration.
Migration + seed; Swagger docs.
Tests for behavior and edge cases.

Stale-path trap: the skill says api/sql/...; this repo uses api/database/.... Check the filesystem.

Teach the team's rules with fictional TAO

Compare fresh Ask chats before and after restoring the repository instructions.

THE SAME QUESTION, BEFORE AND AFTER

How should I add logging and metrics to the supplier route? 5 bullets, no code.

1	2	3	4	5
3 MIN	7 MIN	3 MIN	5 MIN	4 MIN
Before	**Draft**	**Add rule**	**After**	**Compare**
Ask; note the recommendation.	App guidance. 40 lines; one file.	TAO + guide. Never install it.	Fresh Ask chat. Check References.	Compare to main. 3 min buffer.

`.github/copilot-instructions.md`

Done: Ask uses TAO and references your instructions. This is fictional guidance: do not install it.

Repair discovery, then build DeliveryVehicle CRUD

Each vehicle belongs to a branch. Use the skill, then prove the complete feature works.

1 SPOT + READ | 7 MIN

Find the missing header; read the workflow.

2 min to spot the gap + 5 min to read resources.

2 REPAIR + CHECK | 6 MIN

Add name + description; confirm api-endpoint.

5 min for metadata + 1 min to check discovery.

3 BUILD + COMPARE | 12 MIN

Request full CRUD; review the generated files.

10 min to build/verify + 2 min compare; 2 min buffer.

VERIFY THE FEATURE

Model, repository and route.
Register the route.
Migration, seed and Swagger.
Tests + branch relationship.

```
make test-api
make dev-api
port 3000 /api-docs
```

Done: the skill is listed, API tests pass, and the DeliveryVehicle endpoints appear in Swagger.